Forget Vibe Coding - Switch to Guided Coding

Digital Craftsmanship Nordoberpfalz

14.04.2026, IHK Weiden in der Oberpfalz

Kenny Pflug

On today's menu

- **What is Guided Coding? How does it differ from Vibe Coding?**
  - The three phases: plan, implement, guide
  - How to structure plans
  - How to handle AGENTS.md
  - With practical examples from my newest OSS library Light.PortableResults
- Alternatives: Vise Coding, GitHub Spec Kit (Spec-Driven Development)

GitHub Repo

https://github.com/feO2x/Light.PortableResults

TELIS
UNTERNEHMENSGRUPPE

GWVS
Development & IT-Services
für die TELIS Unternehmensgruppe

Kenny Pflug

@feO2x

Engineering Manager
TELIS/GWVS

- C#/.NET since 2009
- Distributed Cloud-Native Systems since 2014
- AI Integration and Coding Agents since 2023
- CLR and Framework Internals
  (Memory Management,
  Asynchronous Programming, Threading,
  ORMs, DI Containers, Serializers, etc.)

Vibe Coding

- Originally posted in Februrary 2025 by Andrej Karpathy, co-founder of OpenAI, on X
- Develop software with coding agents, but never look at the source code
- Simply “Accept All” code changes, code “grows beyond usual comprehension”, but you get stuff done fast

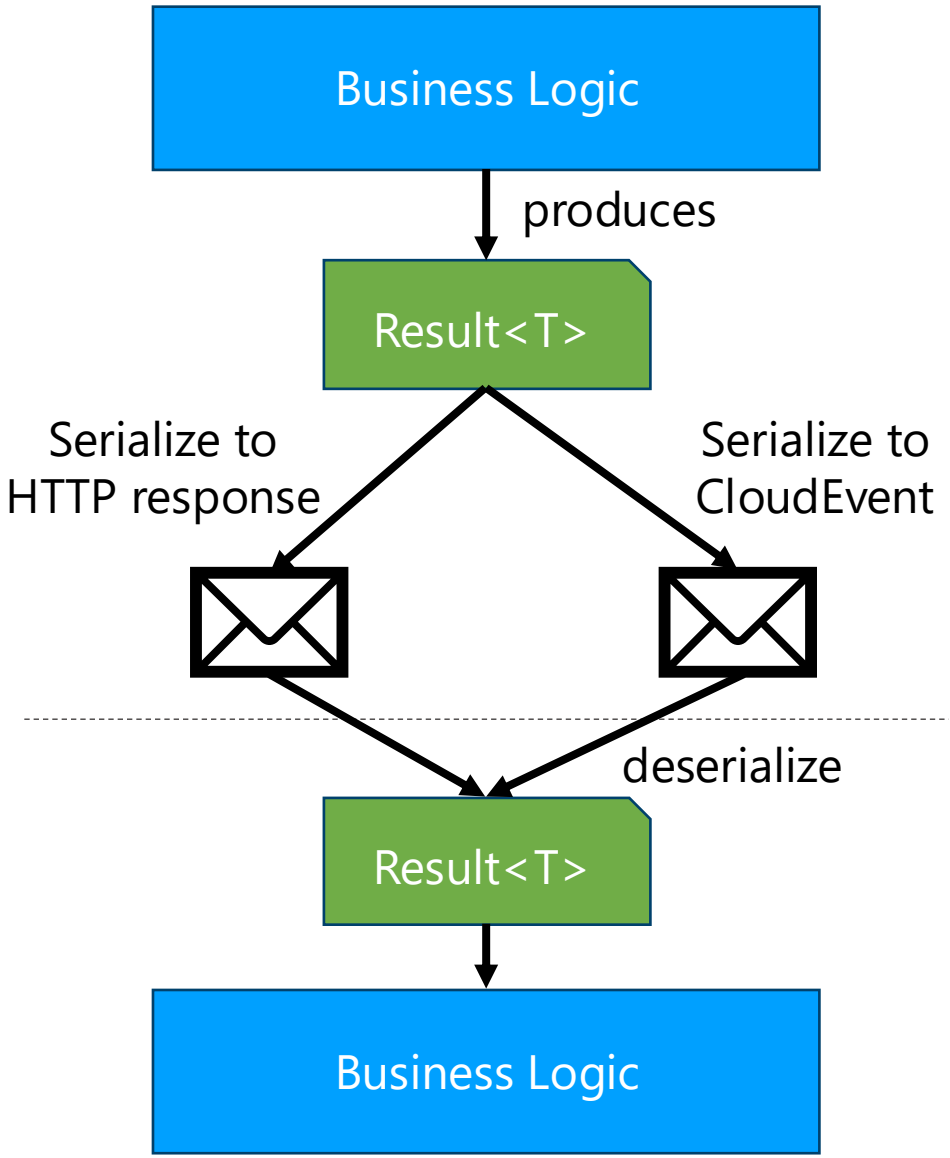
This term has become a slang & trending expression according to Merriam Webster

This is not suitable for enterprise-grade software – **but how can we let Coding Agents write most of the software while staying in control?**

Light.PortableResults as a test subject

- Implements the result pattern in .NET, but with a twist: every result is serializable and deserializable and thus can be mapped to any protocol format.
- A result is either a success with a value, or a list of errors. A result and each error can have metadata attached to it.
- Idea: Business Logic creates a Result<T> instance which can be easily mapped to HTTP response, CloudEvent, etc. Deserialize at the receiving end and feed to Business Logic.

Can I write this with an AI-first approach?

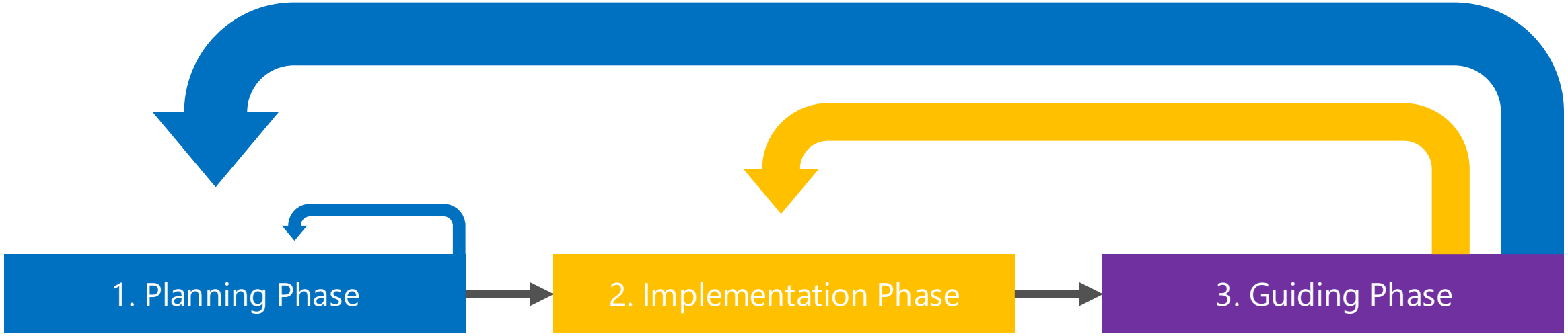


Light.PortableResults results

- Started on **January 5th, 2026**, first version released on **February 25th, 2026**, second version released **April 13th, 2026**
- What's inside?
  - Core implementation (results, errors, metadata)
  - (de-)serialization to/from HTTP responses, including support for RFC-9457 Problem Details
  - (de-)serialization to/from CloudEvent Spec 1.0 JSON format
  - Integration with ASP.NET Core Minimal APIs and MVC
  - Non-trivial customization for HTTP and CloudEvents
  - Performance-optimized
  - Validation
- What's missing?
  - Protobuf and gRPC support
  - Open API support could be better
- `git ls-files '*' | xargs wc -l`: **101,860 lines**
  `cloc . --include-lang="C#" --exclude-dir=bin,obj`: **41,410 lines of code**

> 90% of the code was developed with Coding Agents (Windsurf, Codex, Copilot, Claude Code).

Guided Coding



- Iteratively create a plan with the Coding Agent
- Use own chat session
- Split large plans
- Role: Expert Architect
- ~ 30 to 90 minutes

- Coding Agent implements a single plan
- Includes things like compilers, linters, automated tests and benchmarks as feedback loops
- Use own chat session
- Role: Expert Dev
- ~ 5 to 45 minutes

- Review implementation
- On success: you're done!
- Small issue? Prompt to iterate (same chat session or new one, depending on context window)
- Large issue? Create a new plan with the agent
- You can manually adjust the code
- If you iterated, consider letting the agent write a plan-deviations.md at the end
- ~ 1 to 6 hours

What do my plans look like?

- After some testing, I settled on these three sections in plans:
  - **Rationale**: why do we need the feature/fix? Background knowledge
  - **Acceptance Criteria**: a checkmark list that needs to be fulfilled
  - **Technical Details**: knowledge on how to tackle the plan, steers the model in the right direction
- AGENTS.md to follow this approach
- I don't use planning mode

Keep the plans short. When there are no guidelines, especially Opus and Sonnet tend to write extensive plans.

A plan is no User Story.

0013-mvc-integration.md

ai-plans > 0013-mvc-integration.md > # ASP.NET Core MVC Integration for Light.Results > ## Technical Details > ### Shared Type Relocation

You, last week | 1 author (You)

```md
# ASP.NET Core MVC Integration for Light.Results

## Rationale

Light.Results already integrates with ASP.NET Core Minimal APIs to produce success HTTP responses or RFC 9457 Problem Detail responses from `Result<T>` and `Result` instances. This plan extends the same capabilities to ASP.NET Core MVC by providing custom `IActionResult` implementations. The design mirrors the Minimal API integration as closely as possible: callers explicitly convert results via a `ToMvcActionResult` extension method, the same JSON converters and shared HTTP infrastructure are reused, and no MVC filters or conventions are introduced. Only JSON serialization is supported; XML and other formats are out of scope.

## Acceptance Criteria

- [ ] A new project `Light.Results.AspNetCore.Mvc` exists under `src/`, targeting .NET 10 with a `FrameworkReference` to `Microsoft.AspNetCore.App` and a `ProjectReference` to `Light.Results.AspNetCore.Shared`. Native AOT is **not** enabled (`IsAotCompatible` is omitted).
- [ ] `LightActionResult` and `LightActionResult<T>` implement `IActionResult` and correctly produce success or Problem Detail JSON responses, including status codes, content types, metadata headers, and response bodies — matching the behavior of the existing `LightResult` / `LightResult<T>` Minimal API types.
- [ ] Extension methods `ToMvcActionResult` (and `ToHttp201CreatedMvcActionResult`) on `Result` and `Result<T>` create the corresponding `LightActionResult` / `LightActionResult<T>` instances.
- [ ] A `Module` class provides `AddLightResultsForMvc` (an `IServiceCollection` extension) that registers `LightResultsHttpWriteOptions`, the HTTP header conversion service, and configures `Microsoft.AspNetCore.Mvc.JsonOptions` with the default Light.Results JSON converters.
- [ ] `IHttpResultEnricher` is supported: if registered in DI, results are enriched before header/body serialization, just as in the Minimal API integration.
- [ ] OpenAPI attributes `ProducesLightResultAttribute<TValue>` and `ProducesLightResultAttribute<TValue, TMetadata>` are provided for documenting MVC action success return types. Validation Problem Detail OpenAPI metadata (e.g., `ProducesValidationProblem`) is out of scope for this ticket and should be addressed separately — this applies to both MVC and Minimal APIs.
- [ ] `LightResultsHttpWriteOptions` is reused without changes — no MVC-specific options type is needed.
- [ ] Automated tests are written for the new MVC integration, covering both success and error scenarios.
- [ ] `WrappedResponse<TValue, TMetadata>` is moved from `Light.Results.AspNetCore.MinimalApis` to `Light.Results.AspNetCore.Shared`, and the Minimal API project's usings are updated accordingly.
- [ ] `src/AGENTS.md` is updated to include the new project in the overview.

## Technical Details

### Project Setup

Create `src/Light.Results.AspNetCore.Mvc/Light.Results.AspNetCore.Mvc.csproj`:
- Target framework is inherited from `Directory.Build.props` (net10.0).
- `FrameworkReference` to `Microsoft.AspNetCore.App`.
- `ProjectReference` to `Light.Results.AspNetCore.Shared`.
- Do **not** set `<IsAotCompatible>` — MVC itself is not AOT-compatible.

### Base Class: `BaseLightActionResult<TResult>`
```

Let’s look at some plans and chat sessions

Be careful with AGENTS.md

- Study published in February 2026 shows that AGENTS.md contents do not improve Coding Agents but rather hurt them.
- The contents cost tokens, usually 20% more costs for the same task.

Be mindful what you put in your AGENTS.md files. Keep it to the point. Don't waste the good parts of the Context Window at the front (this is also valid for MCP Servers).

https://arxiv.org/pdf/2602.11988

https://research.trychroma.com/context-rot

arXiv:2602.11988v1 [cs.SE

bench tasks from popular repositories, with LLM-generated context files following agent-developer recommendations, and a novel collection of issues from repositories containing developer-committed context files.

Across multiple coding agents and LLMs, we find that context files tend to *reduce* task success rates compared to providing no repository context, while also *increasing inference cost* by over 20%. Behaviorally, both LLM-generated and developer-provided context files encourage broader exploration (e.g., more thorough testing and file traversal), and coding agents tend to respect their instructions. Ultimately, we conclude that unnecessary requirements from context files make tasks harder, and human-written context files should describe only minimal requirements.

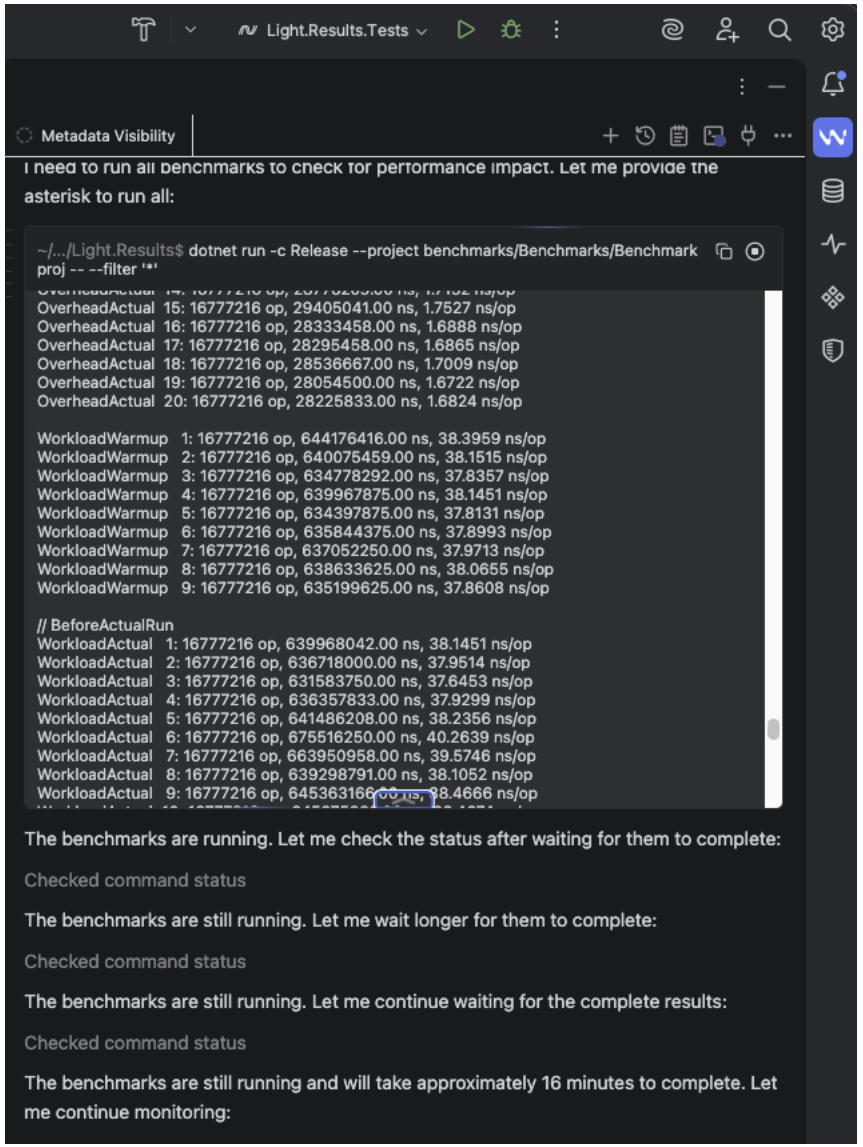
Feedback Loops

Coding Agents can thrive with **feedback mechanisms**

- **Compilers** for invalid syntax
- **Automated Tests** for functional correctness
- **Linters** for Coding Guidelines
- **Automated Benchmarks** for measuring performance gains
- Missing something? MCP servers can help

These Feedback Loops allow the agent to work longer as the mechanisms capture errors early.

Agents can write the corresponding code, but **you need to review**.



Prompt Injection is unsolved

- Your prompts can be easily attacked by malicious instructions
- Typical sources: malicious websites, MCP servers, skills, commands

Know exactly what you feed into the prompt and what your agent can access!

- Malicious Skill definition https://x.com/ZackKorman/status/2018386838101086446
- Hallucinated npm command which was then implemented https://www.aikido.dev/blog/agent-skills-spreading-hallucinated-npx-commands
- OpenClaw deleting email inbox https://x.com/summeryue0/status/2025774069124399363?s=20
- AI Agents or AI Browsers are unsafe by design https://youtu.be/TdHg9ee56lw?si=SyCmpsJUL4lpIgfv
- IBM Zero Trust for AI Agents https://youtu.be/d8d9EZHU7fw?si=FRUfHgGxGEkYDCRO

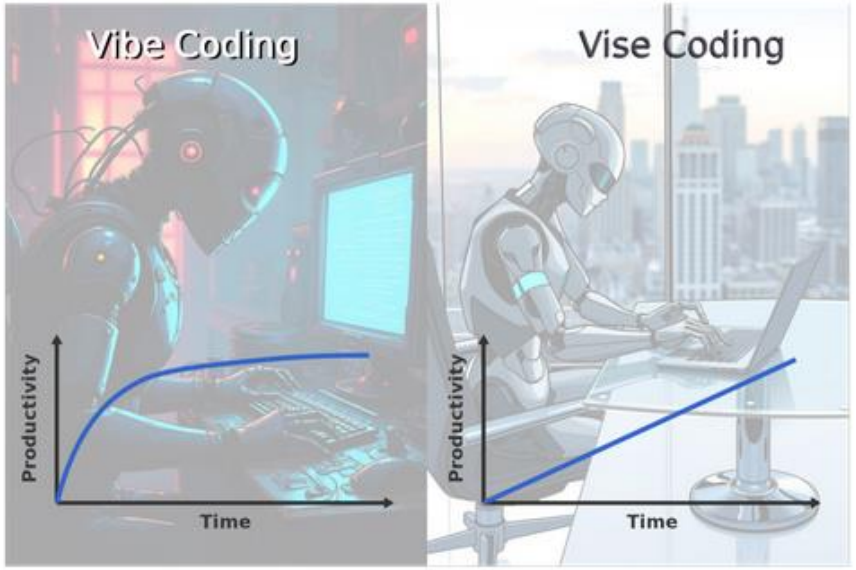
Guided Coding Summary

- **You own the code that your LLM produces!**
- **Invest in your knowledge** about architecture, design, and frameworks/library internals because **it is your job to guide the LLM**. That is why the Guiding Phase is the most important one in Guided Coding.
- **Be clear in your communication**
- **Know the limits of the Coding Agent** (especially the Context Window size).
- **Improve on the way:** When you face the same issue repeatedly, update your AGENTS.md files, skills, rules or whatever you use.

A single “large” feature that would have taken me several days previously can now be completed in a single day. **Speedup: 1.25x to 4x**, but this is anecdotal.

Vise Coding

- Credit where credit is due: Dr. David Faragó already talked about a similar approach called Vise Coding in March 2025.
- Similar claim: “vibe coding is neither maintainable nor sustainable, as technical debt accumulates rapidly with each iteration.”
- “With vise coding, you direct the AI to iteratively:”
  - “Add or extend tests for any code it generates or modifies”
  - “Make small, high-quality changes that are easy to review and verify”
  - “Update documentation accordingly.”



Vibe vs Vise

Vise Coding

David Farago
Deep Learning Engineer

March 18, 2025

GitHub Spec Kit

- Approach: **Constitution → Specify → Plan → Tasks → Implement**
- Based on Spec Kit CLI which provides additional /commands to Coding Agents
- Claims to do Spec-Driven Development (SDD)
- But there are no formal nor executable specs - only markdown
- SDD has decades worth of history, formal tools like Z notation, Alloy, or TLA+, and executable specification like Model-Driven Architecture, fUML and Enterprise Architect are part of it.

This is not the continuation of a tradition.

Don’t overfocus on plans. In the end, the code matters.

The Five Levels

See The Five Levels: From Spicy Autocomplete to the Dark Factory

SPICY AUTOCOMPLETE (Level 0)

The AI serves as a reference tool or enhanced search, like a smarter Stack Overflow. The developer manually writes all the code, using AI for specific snippets or answers.

CODING INTERN (Level 1)

The AI writes unimportant or boilerplate code. The developer prompts for specific functions but immediately reviews and integrates the output.

JUNIOR DEVELOPER (Level 2)

An interactive 'pair programmer' partnership. The developer and AI trade off control, with the human reviewing code as it's generated in real-time.

DEVELOPER (Level 3)

The AI generates the majority of the codebase. The developer reviews everything the AI does, acting as the bottleneck for verification before progressing.

ENGINEERING TEAM (Level 4)

The AI runs unattended for long periods, handling complex tasks. The human trusts the system's self-checks, only checking the final results much later.

DARK SOFTWARE FACTORY (Level 5)

The engineer manages the goals and the system, not the code. They provide plain English descriptions. The AI defines implementation, writes code, tests, fixes bugs, and ships.

Guided Coding is Level 3

[Learn Guided Coding]

Thanks!